BİTİRME PROJESİ · BÖLÜM 2 · IMPLEMENTASYON

Karpathy modelleri FPGA’de

Dokuz language model mimarisini tamamen integer, bilgisayarsız çalışan tek bir FPGA tasarımında koşturmak: aritmetik, donanım, doğrulama ve yerleşim

Murat Atceken

Nexys A7-100T · Artix-7 XC7A100T · Ekim 2026

ÖZET

Sonuç: 18 model tek bir integer engine’de, bilgisayarsız ve Python referansıyla bit bit aynı çalışıyor

9 → 18

mimari → model

bigram’dan nanochat’e 9 mimari; Karpathy’nin stories260K’sı, 5 TinyStories sürümü ve 3 düşük bitli model.

1

ortak RTL engine

Modelden modele değişen yalnızca BRAM’deki mikrokod ve weight’ler.

18 / 18

bit-exact

Tam kart simülasyonunda her token, ekran ve VGA karesi referansla aynı.

0

floating point

Softmax, norm ve sampling dahil bütün hesap integer.

Somut örnek

Karpathy’nin 260 bin parametreli stories260K modeli, kartın VGA ekranına 256 token’lık bir hikâye yazıyor. Simülasyonda üretilen her token, Python’daki integer modelin ürettiğiyle aynı; token başına 50 397 cycle.

YOL HARİTASI

Sunum, bir modelin PyTorch’tan çipe giden yolunu dokuz adımda izliyor

1 · Hedef
Bilgisayarsız çalışan kart ve kısıtları

2 · Model ailesi
9 mimari, stories260K, TinyStories

3 · Integer aritmetik
quantization, requant, softmax, norm

4 · Derleyici
PyTorch → mikrokod → bellek imajları

5 · Engine
zoo_core: GEMV, attention, norm

6 · Kart sistemi
üretim döngüsü, sampling, VGA

7 · Düşük bit
Larq, LittleBit, 1-bit GEMV

8 · Doğrulama
8 halkalı zincir, bulunan hatalar

9 · Yerleşim ve hız
Fmax, kaynaklar, token/s

1 · HEDEF

Amaç: eğitilmiş bir language model’i FPGA’nin içinde, bilgisayara bağlı olmadan çalıştırmak

Host sürücülü düzen (seçilmedi)

PC: döngü, sampling, metin ⇄ UART ⇄ **FPGA: tek adım**

PC her token için komut gönderir, logit’leri okur, bir sonraki token’ı kendisi seçer. FPGA yalnızca bir hesaplayıcıdır; model “PC’de çalışıyor” gibi görünür.

Bağımsız düzen (bizim tasarımımız)

FPGA: weight’ler + döngü + sampling + metin → VGA

Weight’ler bitstream ile BRAM’e gömülür. Üretim döngüsü, sampling, token’ı metne çevirme ve ekrana yazma çipin içindeki mantıkta çalışır.

Yalnız inference
Eğitim ve QAT Mac’te yapılır; FPGA’de eğitim yok.

Yalnız Karpathy modelleri
makemore, minGPT, microgpt, llama2.c, nanochat.

Doğrulama Mac’te
Kartın tamamı simüle edilir; fiziksel kart testi yok.

1 · HEDEF

Hedef kart Nexys A7-100T: işlemcisi olmayan, orta büyüklükte bir Artix-7 FPGA kartı



XC7A100T kaynağı	sayı	işi
LUT (6 girişli)	63 400	mantık
flip-flop	126 800	register
BRAM (36 Kb)	135	~607 KB bellek
DSP48E1	240	25 × 18 çarpıcı
işlemci	yok	–

Şekil: solda kullandığımız giriş/çıkışlar, ortada FPGA. Kartın 128 MB DDR2’si kullanılmıyor; model yalnız çip içi BRAM’de duruyor.

1 · HEDEF

Tasarımı dört kısıt şekillendirdi: işlemci yok, floating point yok, dış bellek yok, tek clock

İşlemci yok

Üretim döngüsü, sampling, detokenizer ve ekran sürücüsü birer donanım FSM’i olarak yazıldı.

Floating point yok

DSP48 bir integer çarpıcı. Bütün hesap INT8 ve INT16; exp, karekök ve bölme de integer.

Dış bellek yok: model BRAM’e sığmalı

stories260K: 260 bin weight × 1 bayt ≈ 254 KB, KV cache 5 × 256 × 32 × 2 bayt = 80 KB. Toplam 607 KB’ın altında. GPT-2 124M ise 124 MB isterdi.

Tek clock

100 MHz → MMCM → çekirdek clock’u 50 ya da 25 MHz. VGA’nın 25 MHz piksel hızı aynı clock’tan bir enable ile türetilir.

Bu kısıtların sonucu

Modeller küçük (50 bin – 260 bin parametre) ama tasarım eksiksiz: bir bilgisayarın yaptığı her işi çip kendisi yapıyor. Dış bellek olmadığı için bellek gecikmesi de yok; her cycle önceden bilinir.

2 · MODEL AİLESİ

Dokuz mimari, Karpathy’nin dört deposundan geliyor ve aynı ~50 bin parametre bütçesiyle eğitildi

model	kaynak	norm	activation	pozisyon / durum	attention	param.
bigram	makemore	–	–	yalnız son token	–	729
mlp	makemore	–	tanh	16 token pencere	–	50 459
rnn	makemore	–	tanh	recurrent state	–	49 347
gru	makemore	–	σ, tanh	gated state	–	48 823
bow	makemore	–	tanh	causal ortalama	–	49 979
gpt2	minGPT, nanoGPT	LayerNorm	GELU	learned	MHA	53 120
microgpt	microgpt gist	RMSNorm (param.siz)	ReLU	learned	MHA	51 392
llama	llama2.c	RMSNorm (gain)	SwiGLU	RoPE	GQA	50 304
nanochat	nanochat	RMSNorm (param.siz)	ReLU²	RoPE	GQA, QK-norm	46 784

Hepsi names.txt’te (32 bin isim, 27 karakter) eğitildi. Gri satırlar transformer’lar: 4 layer, C = 32, 16 token context.

Mimariler aynı birkaç yapı taşının farklı birleşimleri; engine bu yapı taşlarını birer op olarak sunuyor

model	EMB	GEMV	NORM	ATTN, KV	ROPE	MUL	state	BOW	op sayısı
bigram	•								1
mlp	•	•							3
rnn	•	•					•		5
gru	•	•				•	•		8
bow	•	•						•	7
gpt2	•	•	•	•					43
microgpt	•	•	•	•					44
llama	•	•	•	•	•	•			59
nanochat	•	•	•	•	•				60
stories260K	•	•	•	•	•	•			73

Tablo: mikrokodda cycle harcayan op türleri (Verilator profili); state = COPY, GRUUPD, HRESET.

stories260K, Karpathy'nin TinyStories'te eğittiği gerçek bir Llama 2; diğer mimarileri de aynı boyda eğittik

stories260K (llama2.c, hazır checkpoint)

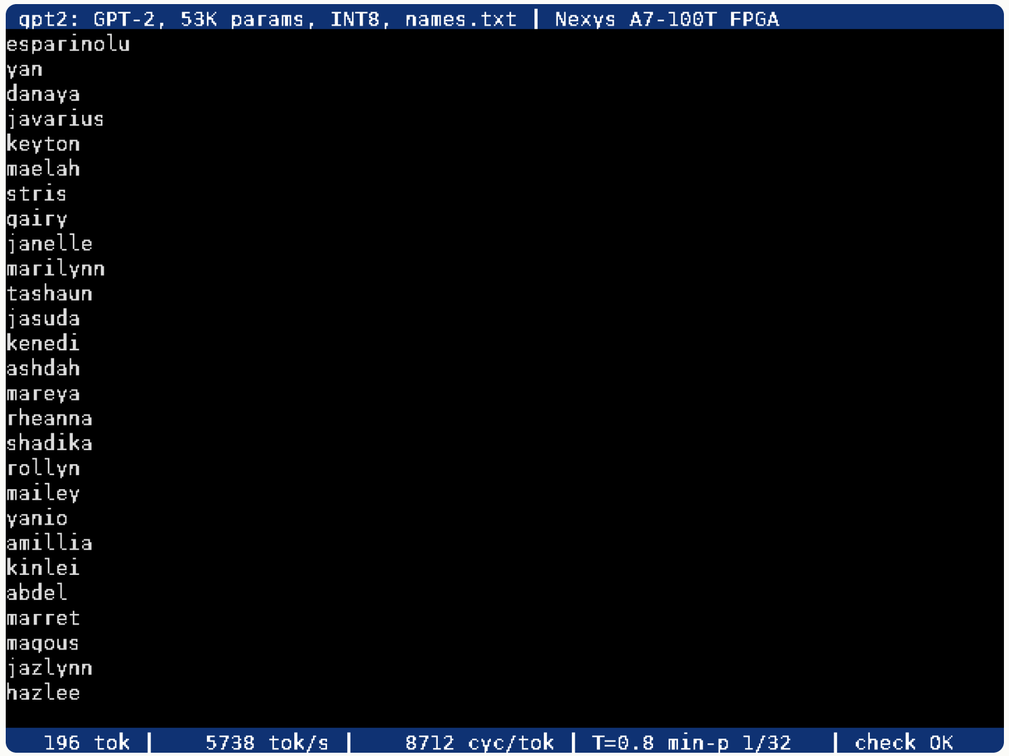
özellik	değer
embedding boyutu	64
layer	5
head / KV head	8 / 4
vocabulary	512 token
context (kartta)	256
loss: float → INT8	1,400 → 1,422

Aynı veride bizim eğittiğimiz sürümler (ts_*)

mimari	parametre	test loss
ts_llama	260 032	1,346
ts_gpt2	259 728	1,438
ts_microgpt	261 120	1,575
ts_gru	257 920	1,610
ts_nanochat	260 096	1,637

Float test loss (nat/token), kendi eğitim ve test düzenimizle. stories260K'nın loss'u 400 doğrulama hikâyesinde ölçüldü; iki sütun doğrudan karşılaştırılmamalı.

Kartın ekranından: aynı devre bir modelle isim, diğeriyle hikâye yazıyor



gpt2, names.txt: 196 token, 8 713 cycle/token



stories260K, TinyStories: 256 token, 50 397 cycle/token

3 · INTEGER ARİTMETİK

Neden integer: aynı çarpma-toplamayı bir DSP tek cycle’da yapıyor ve model dört kat küçülüyor

float32: a·b + c

1. üsleri topla, mantisleri çarp
2. c ile üsleri hizala (kaydır)
3. topla, sonucu normalize et
4. yuvarla, özel durumları ele al

Çipte hazır bir float birimi yok; bunlar LUT’larla kurulur, pahalı ve yavaştır.

INT8: a·b + c

```
x = [12, −7, 30]
w = [18, −55, 127]
Σ wᵢxᵢ = 216 + 385 + 3 810 = 4 411
```

Her terim bir DSP48’de tek cycle: 8 × 8 bit çarp, 32 bitlik akümülatöre ekle.

Bellek: stories260K

BRAM · 607 KB

float32 · 1 040 KB (sığmaz)

INT8 · 254 KB

Bedeli

Her sayı artık “scale × tamsayı”. Soru şu: scale’leri nasıl seçeriz ve tamsayılar arasındaki işlemleri float’a hiç dönmeden nasıl yaparız? Sonraki beş slayt bunun cevabı.

Quantization: her gerçel sayı, bir scale ile çarpılmış bir tamsayı olarak yazılır

Gerçek bir weight satırı: max |w| = 0,268

```
S = 0,268 / 127 = 0,00211
```

w	w / S	q	S · q	hata
0,0372	17,63	**18**	0,0380	+0,0008
−0,1153	−54,64	**−55**	−0,1161	−0,0008
0,2680	127,00	**127**	0,2680	0
0,0009	0,43	**0**	0	−0,0009

Hata hiçbir zaman S/2 = 0,00106'yı geçmez. Çok küçük weight'ler sıfıra yuvarlanır.

$$q = \text{clip}(\text{round}(r / S), -127, 127)$$
$$\hat{r} = S \cdot q, \; |r - \hat{r}| \le S/2$$

Weight'ler: çıkış kanalı başına S

Her satırın kendi max |w|'si; büyük satır küçük satırı ezmez.

Activation'lar: tensör başına sabit S

Kalibrasyon verisinde |x|'in %99,99 yüzdeliğinden. Uç değerler kırpılır, kalan değerler daha ince çözünürlük kazanır.

Requant: gerçel çarpan M, bir tamsayı çarpan ve bir kaydırma olur; float'a hiç dönülmez

$$y \approx S_x\, S_w \cdot \mathrm{acc},\ \mathrm{acc} = \Sigma\, q_w\, q_x + q_b$$
$$q_y = \mathrm{round}(M \cdot \mathrm{acc}),\ M = S_x\, S_w \,/\, S_y$$
$$M \approx m_0 \cdot 2^{-\mathrm{sh}},\ m_0 \in [2^{15}, 2^{16})$$

$$q_y = \mathrm{sat}_8(\ (\mathrm{acc} \cdot m_0 + 2^{\mathrm{sh}-1}) \gg \mathrm{sh}\ )$$

Bağıl hata ≤ 2^{-16}. m_0 ve sh her çıkış kanalı için parametre ROM'unda durur; donanımda bir çarpma, bir toplama ve bir kaydırma.

Sayısal örnek

```
S_x = 0,05 S_w = 0,00211 S_y = 0,077
M = 0,05 · 0,00211 / 0,077
= 0,0013703
sh = 25, m0 = round(M · 2²⁵) = 45 979

acc = 5 321
acc · m0 = 244 654 259
+ 2²⁴ = 261 431 475
» 25 = 7
float: 5 321 · M = 7,29 → 7
```

Integer softmax: üstel fonksiyon bir kaydırma ile 256 satırlık bir tabloya bölünür, bölme bir kez yapılır

$$1 \cdot m = \max s_j,\ u_j = (m - s_j) \cdot C$$

$$2 \cdot u_j = 256\, n_j + f_j,\ e_j = \mathrm{LUT}[f_j] \gg n_j,\ \mathrm{LUT}[f] = 2^{15} \cdot 2\text{^}(-f/256)$$

$$3 \cdot E = \Sigma\, e_j,\ R = \lfloor 2^{32} / E \rfloor,\ y = (\Sigma\, e_j\, v_j) \cdot R \gg 32$$

Neden işe yarıyor: $e^x = 2\text{^}(x \cdot \log_2 e)$. 2'nin tam kuvveti bir kaydırma; kesirli kısım tablodan. Max çıkarıldığı için her üs ≤ 0, taşma yok. Normalizasyon en sona ertelenir: head başına tek bölme.

Hata: kesirli kısmın 8 bite kesilmesi %0,27'den az bağıl hata getirir.

Örnek: üç score, C = 92

s	u	n, f	e	e / E	float
9	0	0, 0	32 768	0,607	0,607
6	276	1, 20	15 520	0,287	0,287
2	644	2, 132	5 730	0,106	0,106

E = 54 018, $R = \lfloor 2^{32} / 54\,018 \rfloor$ = 79 509. Üç olasılık, float softmax ile üç basamakta aynı.

Integer LayerNorm ve RMSNorm: vektör başına bir karekök ve bir bölme; çıkışın taşmayacağı ispatlı

SUM

$s = \Sigma\ x_i$

VAR

$d_i = N \cdot x_i - s$

$Q = \Sigma\ d_i^2 + E$

SQRT

$rt = \lfloor \sqrt{Q} \rfloor$

sıralı, bit bit

DIV

$inv = \lfloor K / rt \rfloor$

sıralı bölücü

OUT

$z_i = (d_i \cdot inv) \gg k$

Tam integer

$d_i = N(x_i - \mu)$ bölmesiz ve kayıpsız. RMSNorm'da $s = 0$ alınır; gerisi aynı birim.

Taşma yok (Samuelson)

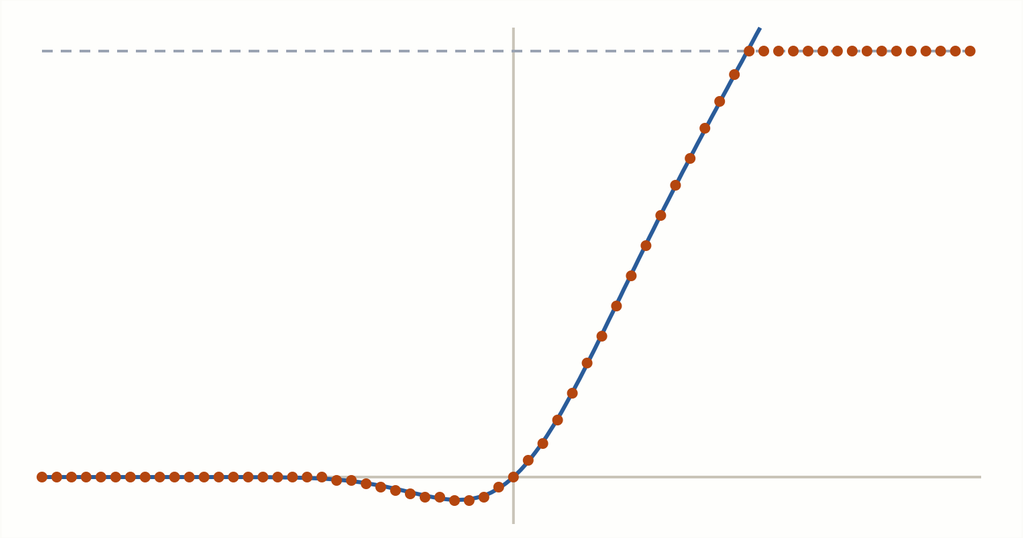
$|x_i - \mu| \le \sigma\sqrt{N - 1}$. Çıkış formatı bu sınıra göre seçilir; kalibrasyon gerekmez, doyma olmaz.

γ ve β katlanır

$W(\gamma \odot \hat{z} + \beta) + b = (W \cdot \mathrm{diag}\ \gamma)\ \hat{z} + (W\beta + b)$. Donanım yalnız $\hat{z}$'yi hesaplar.

Maliyeti: NORM, gpt2'de cycle'ların %14'ü; nanochat'te QK-norm yüzünden %33'ü. Karekök ve bölme yavaş ama vektör başına yalnız bir kez çalışıyor.

INT8 girişli her fonksiyon 256 satırlık bir tablodur: GELU, SiLU, tanh, σ ve ReLU² yaklaşımsız



Şekil: mavi: gerçek GELU; turuncu: tablonun her 4. satırı (q = −128… 127, S_h = 0,05, S_g = 0,025). Kesikli çizgi: 127'de doyma.

q	x = q·S_h	GELU(x)	tablo
−40	−2,0	−0,045	−2
−10	−0,5	−0,154	−6
0	0	0	0
20	1,0	0,841	34
60	3,0	2,996	120

Giriş yalnızca 256 farklı değer alabildiği için tablo fonksiyonu **tam** temsil eder. Hata yalnız giriş ve çıkış quantization'ından gelir.

Engine'de tablo GEMV'nin çıkışına takılır: requant → LUT → INT8. Katman başına 256 bayt.

3 · INTEGER ARİTMETİK

Bedel küçük: INT8'e geçmek names modellerinde en fazla 0,006, stories260K'da 0,022 nat kaybettiriyor

model	float	INT8	Δ loss (nat/token)	Δ	argmax aynı
bigram	2,451	2,451		0,0000	%100,0
mlp	1,986	1,987		+0,0011	%98,1
rnn	1,964	1,967		+0,0032	%96,1
gru	1,956	1,956		+0,0003	%98,3
bow	2,045	2,045		−0,0003	%97,0
gpt2	2,000	2,002		+0,0017	%96,3
microgpt	1,971	1,974		+0,0032	%95,3
llama	2,003	2,004		+0,0013	%93,5
nanochat	1,969	1,975		+0,0055	%93,8
stories260K	1,400	1,422		**+0,0217**	%92,0

Float model ile donanımla bit-exact integer program, aynı test verisinde. Son sütun: en olası token'ın float ile aynı çıktığı adımların oranı.

4 · DERLEYİCİ

Derleyici, bir PyTorch modelini altı adımda donanımın okuyacağı mikrokoda ve bellek imajlarına çevirir

1 · Model
eğitilmiş .pt ya da Karpathy'nin .bin'i

→

2 · IR
op listesi; PyTorch ile fark < 10^{-4}

→

3 · Katlama
γ, β, $1/\sqrt{d_h}$ weight'lere

4 · Kalibrasyon
activation scale'leri, %99,99

→

5 · Integer program
IntRunner: altın referans

→

6 · Export
.mem dosyaları ⟶ BRAM

dosya	içerik	gpt2'de
prog.mem	op kelimeleri (mikrokod)	43 op
weights.mem	INT8 weight'ler, 8'li şeritler hâlinde	6 272 × 64 bit
params.mem	kanal başına bias, m_0, sh	1 184 satır
tables.mem, luts.mem	embedding'ler, exp ve activation tabloları	1 376 + 1 024

4 · DERLEYİCİ

Mikrokod, modelin forward fonksiyonunun op op yazılmış hâli: gpt2 ile llama’nın bir block’u

gpt2 · layer 0 (43 op’un 10’u)

```
NORM ln R → A
GEMV q A → Q
GEMV k A → K
GEMV v A → V
KVSTORE layer 0
ATTN 4 head, d=8 → B
GEMV proj B → R += residual
NORM ln R → A
GEMV fc A → F GELU tablosu
GEMV mproj F → R += residual
```

llama · layer 0 (59 op’un 14’ü)

```
NORM rms R → A
GEMV q, k, v A → Q, K, V
ROPE q: 4 head
ROPE k: 2 head
KVSTORE layer 0
ATTN 4 head / 2 KV head
GEMV wo B → R += residual
NORM rms R → A
GEMV w1 A → F SiLU tablosu
GEMV w3 A → G
MUL F ⊙ G → F SwiGLU
GEMV w2 F → R += residual
```

Turuncu satırlar iki mimarinin farkı: RoPE, grouped-query attention ve SwiGLU’nun eleman bazlı çarpımı. Geri kalan op’lar aynı; engine ikisini de aynı devreyle çalıştırır.

4 · DERLEYİCİ

En büyük model stories260K bile BRAM'in yaklaşık %71'ini kullanıyor; en büyük pay weight'lerin

stories260K'nın çip içi belleği (607 KB = tüm çubuk)



■ weight'ler 256 KB ■ KV cache 80 KB ■ tablolar (embedding, exp) 68 KB ■ requant parametreleri 28 KB ■ mikrokod, LUT'lar 3 KB

□ boş ~172 KB

model	BRAM36	kapasitenin
bigram	2	%1
gpt2 (names)	19,5	%14
ts_gru	69,5	%51
stories260K	97,5	%72
stories260K-bin (1 bit)	56	%41

Weight'ler bitstream'in parçası: **$readmemh** ile okunan .mem dosyaları BRAM'lerin başlangıç içeriği olur. KV cache ve vektör tamponları ise çalışırken yazılan RAM'ler. Sayılar Yosys sentezinin tahmini.

5 · ENGİNE

zoo_core: mikrokodu op op okuyan, her op'u kendi birimine gönderen tek bir engine

prog.mem → fetch, decode: op, tamponlar, boyut, adres → op bitince sıradaki; END'de token hazır

BRAM

weights · 8 × INT8 / kelime

wbin · 64 × 1 bit / kelime

params · bias, m_0, sh

tables, luts

KV cache (yazılır)

GEMV
8 MAC hattı, DSP

GEMVB
64 ± hattı, 1 bit

requant
seri, + LUT

NORM
LN, RMS, √, ÷

ATTN, KVSTORE
8 hat, exp tablosu

ROPE
sin/cos tablosu

EMB, TABLOGIT
tablo okuma

MUL, REQ, SCALE
eleman bazlı

COPY, GRUUPD, BOW
state, ortalama

Tamponlar

11 vektör tamponu

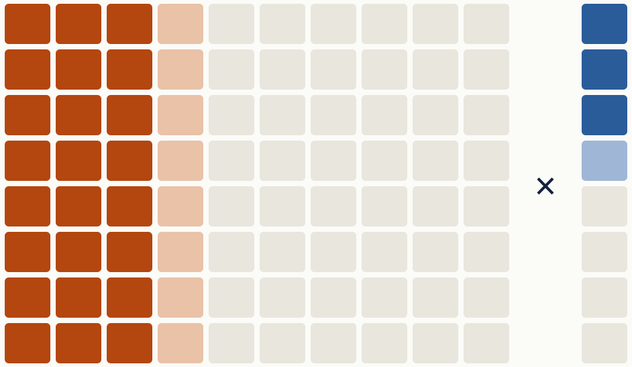
residual: INT16

diğerleri: INT8

logit'ler: INT16 → lm_gen

5 · ENGİNE

GEMV: her cycle’da bir giriş 8 hatta yayınlanır ve 8 çıkış aynı anda birikir



Şekil: 8 satırlık bir çıkış bloğu (turuncu) ve giriş vektörü (mavi). Koyu sütunlar işlenmiş girişler; açık olan bu cycle’ınki: x_i 8 hatta yayınlanır, 64 bitlik tek bir weight kelimesi 8 weight’i birden getirir.

$$\text{cycle} \approx \lceil n_out / 8 \rceil \cdot n_in + \text{sabit}$$

gpt2 fc (32 → 128): 16 blok × 32 = 512 cycle.
stories260K w1 (64 → 172): 22 blok × 64 = 1 408 cycle.

Blok bitince 8 akümülatör bir gölge bankasına kopyalanır ve requant’a seri akar; hatlar hemen sonraki bloğa başlar. Bu yüzden cycle sayısı önceden hesaplanabilir.

5 · ENGİNE

Attention birimi: her geçmiş token, her head’de iki cycle ekler; bu yüzden maliyet context ile doğrusal büyür

SCORE

$s_j = q \cdot k_j$, 8 çarpıcı, cycle başına bir key

t + 1 cycle

EXP

e_j = LUT[f] ≫ n, E += e_j

score’larla örtüşür

DIV

$R = \lfloor 2^{32} / E \rfloor$, head başına bir kez

sabit

AV

$y = (\Sigma\, e_j\, v_j) \cdot R \gg 32$, 8 hat

t + 1 cycle

stories260K: token başına cycle, pozisyona göre

t = 0 · 38 652



t = 255 · ~59 000



t = 511 · 79 532

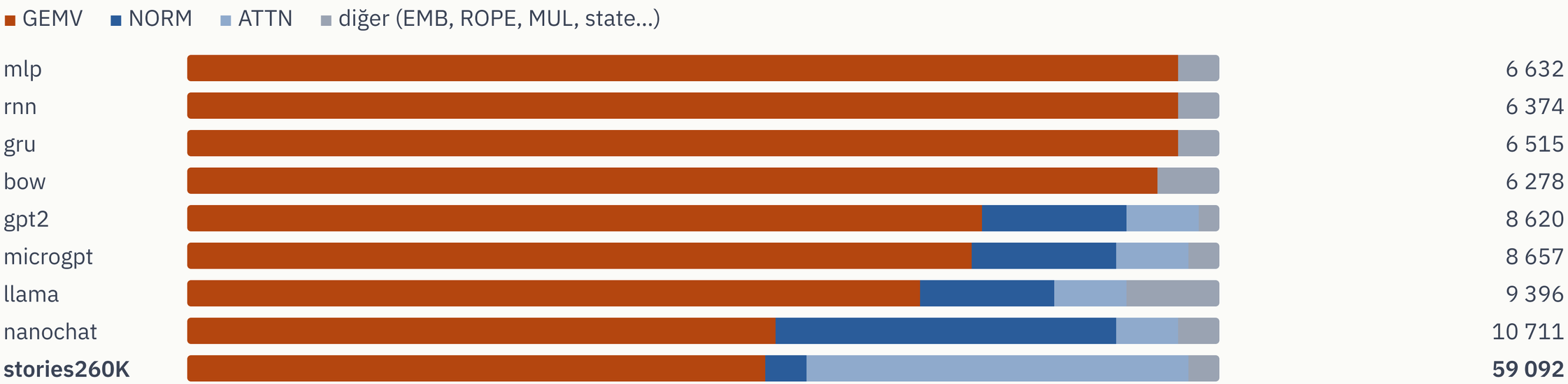


eğim = 5 layer × 8 head × 2 = 80 cycle / token

(79 532 − 38 652) / 511 = 80,0. Ölçüm modelle birebir. Kartta context’i 256 ile sınırlamak KV cache’i 80 KB’ta tutuyor ve en uzun adımı ~59 000 cycle’da bırakıyor.

5 · ENGİNE

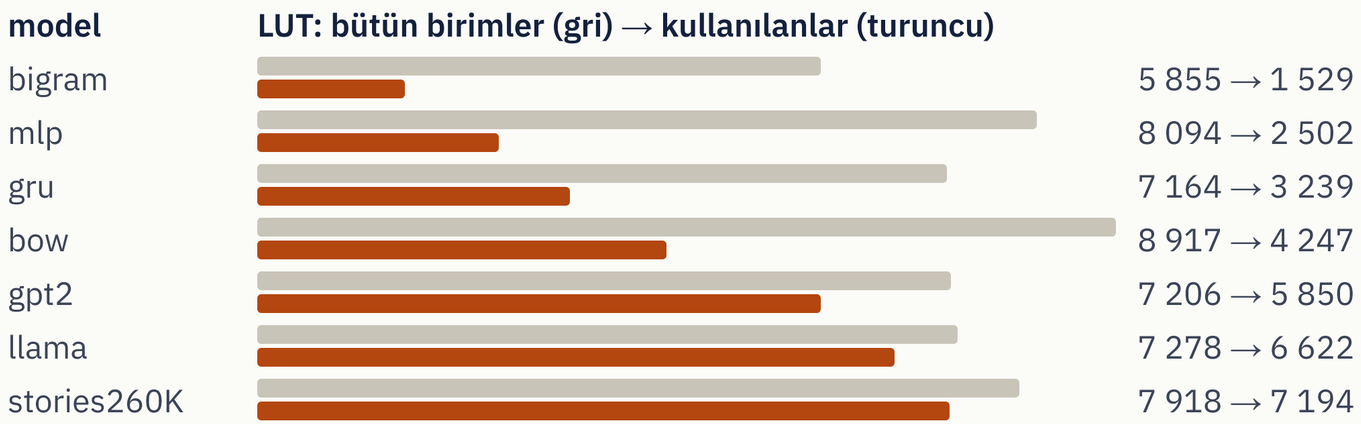
Cycle’lar nereye gidiyor: makemore’da neredeyse hepsi GEMV; transformer’larda norm ve attention da pay alıyor



Çubuklar: op türlerinin token başına cycle içindeki payı (Verilator profili); sağda engine’in token başına ortalama cycle’ı. stories260K: T = 512, bütün pozisyonların ortalaması.

5 · ENGİNE

SPECIALIZE: modelin kullanmadığı birimler sentezde atılır; makemore modellerinde engine dörtte birine iner



Mekanizma (zoo_core.sv)

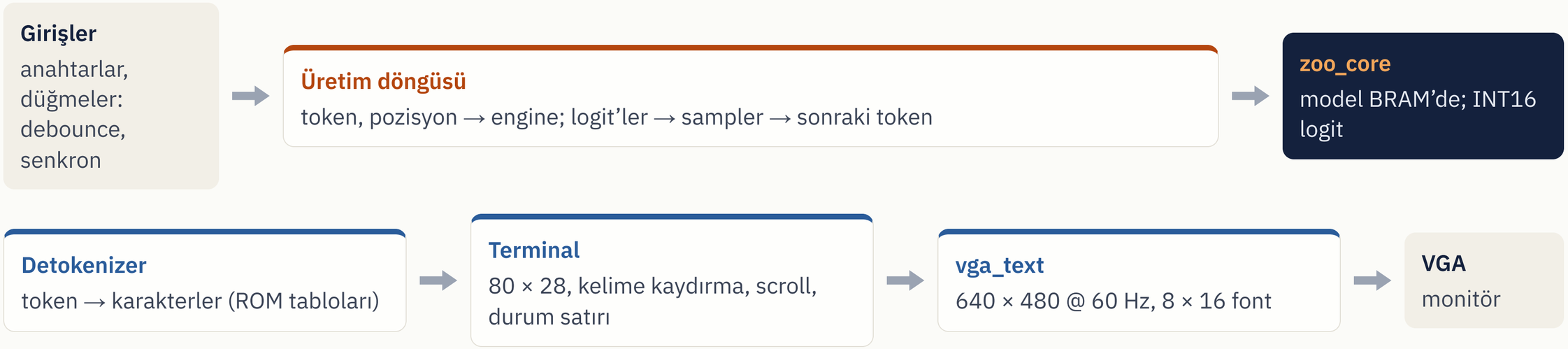
```
EN = SPECIALIZE ? OPS_USED
: 32'hFFFF_FFFF;
OP_NORM: if (EN[6]) ...
```

Derleyici her model için hangi op'ların kullanıldığını bir bit maskesine yazar. Maskede sıfır olan birimin mantığı sabit olarak kapalıdır ve sentez aracı onu tamamen siler.

Yalnız engine, Yosys tahmini. Bütün birimli tek bitstream'lik sürüm: 9 444 LUT.

6 · KART SİSTEMİ

Engine’in etrafında, bir bilgisayarın yapacağı her işi yapan donanım: döngü, sampling, metin ve ekran



Kart üstü kontrol

Token’ların hash’i, Python’un hesaplayıp bitstream’e gömdüğü değerle karşılaştırılır.

Örtüşme

Ekran yazımı sonraki token’ın hesabıyla paralel yürür; yalnız bigram ekranı bekler.

7-segment, LED

token/s ya da cycle/token; token sayacı, kontrol sonucu.

6 · KART SİSTEMİ

Üretim döngüsü bir durum makinesi: engine’i çalıştırır, logit’leri üç kez tarar ve bir token seçer

G_STEP → **G_WAIT** → G_MAX → G_SUM → G_SEL → G_PICK → G_STEP …

durum	iş	cycle
G_STEP, G_WAIT	engine’e (token, pozisyon) ver, END’i bekle	engine’in cycle’ı
G_MAX	en büyük logit m; greedy ise argmax ile doğrudan G_PICK	V
G_SUM	$e_k = 2^{(-(m - l_k)c)}$, min-p kesmesi, Σe	V
G_SEL	thr = (r · Σe) ≫ 32; birikimli toplam thr’yi aşınca seç	≤ V
G_PICK	token’ı ekrana gönder, hash’i güncelle, pozisyonu artır	1

Sampler’ın ek maliyeti ≈ 3V cycle: stories260K’da 3 × 512 ≈ 1 500 cycle, yani token başına 50 397 cycle’ın %3’ü; names modellerinde ~80 cycle.

Integer sampling: temperature ve min-p, softmax ile aynı tabloyla ve hiç bölme yapmadan

$e_k = 2\text{^}(-(m - l_k) \cdot c)$, $c = s_\text{logit} \cdot \log_2 e / \tau$

kes: $e_k < 2^{15} \cdot 2\text{^}(-\text{NCUT})$ (min-p)

$\text{thr} = (r \cdot \Sigma\, e_k) \gg 32$, r: xorshift32

Neden bölme yok: olasılıkları normalize etmek yerine eşik toplamla ölçeklenir. Rastgele sayı 32 bitlik bir kesir gibi davranır: thr / Σe = r / 2^{32}.

τ anahtarlarla 8 değerden, min-p 4 değerden seçilir; c her τ için sabit noktada saklanır.

Örnek: 4 token, s_logit = 0,05, τ = 0,8, min-p = 1/32

l_k	u	e_k	integer p	float p (min-p)
120	0	32 768	0,7775	0,7773
100	462	9 379	0,2225	0,2227
60	1 385	770 → kes	0	0
20	2 308	63 → kes	0	0

Σe = 42 147. r = 0,75 · 2^{32} → thr = 31 610: ilk token (32 768 > thr). r = 0,875 · 2^{32} → thr = 36 878: ikinci token.

Ekran: token’lar karakterlere çevrilir, kelime bölünmeden yazılır ve VGA’ya her karede yeniden çizilir

Detokenizer

Her token için iki ROM: karakter dizisinin başı ve uzunluğu, sonra baytlar. stories260K’nın 512 token’ı 1 ile birkaç karakter arası.

Terminal

Bir kelime 80. sütunu aşacaksa yarım kelime alt satıra taşınır. Ekran dolunca satırlar dairesel tamponda kayar (scroll).

vga_text

640 × 480 @ 60 Hz, 25 MHz piksel hızı; 80 × 30 karakter, 8 × 16 font. Üç kademeli pipeline: metin RAM’i → font ROM’u → piksel.

Durum satırı

token/s = f_clk / cycle_per_token sıralı bir bölücüyle; sayılar bin2bcd ile onluğa çevrilir.



Şekil: pinlerden yakalanan kare. Satır sonlarında kelimeler bölünmemiş.

6 · KART SİSTEMİ

Kullanım: anahtarlar üretimi ayarlar, düğmeler koşu başlatır; LED16 kartın Python ile aynı sonucu bulduğunu söyler

giriş	işlev
güç, CPU_RESET	ilk koşu kendiliğinden başlar
BTNC	yeni koşu; rastgele durum sürer
BTNU	üreteci SW[15:8]'den tohumla ve koş
BTND	metin alanını temizle
SW0	greedy ya da sampling
SW[3:1]	τ: 0,8 · 1,0 · 0,9 · 0,7 · 0,6 · 0,5 · 1,2 · 1,5
SW[5:4]	min-p: 1/32 · kapalı · 1/128 · 1/16
SW6, SW7	daktilo modu (~25 token/s); 7-segment: token/s ya da cycle/token

Kart üstü kontrol

`h ← 31 · h + tok`

Python, export sırasında greedy koşunun ve ilk sampling koşusunun hash'ini hesaplayıp bitstream'e gömer. Kart kendi hash'ini bununla karşılaştırır.

mavi: üretiyor | **yeşil: OK** | **kırmızı: FAIL**

7 · DÜŞÜK BİT

1 bitlik weight, belleği 8 kat küçültür ve çarpmayı toplama-çıkarmaya çevirir

INT8

```
x = [12, −7, 30]
w = [18, −55, 127]
Σ wx = 4 411 3 çarpma
```

binary: w ≈ a · sign(w)

```
sign = [+, −, +]
12 + 7 + 30 = 49
y = a · 49 0 çarpma (+ 1 scale)
```

Hocamızın önerdiği iki kaynak

LittleBit (Lee vd., NeurIPS 2025): low-rank binary factor'larla 1 bitin altı.

Larq: binary neural network kütüphanesi; quantizer'ları Bi-Real Net, TWN.

Kural değişmedi

FPGA'de eğitim yok. Düşük bitli weight'ler Mac'te QAT ile öğrenilir; FPGA yalnızca sonucu çalıştırır. Embedding ve output head INT8 kalır.

7 · DÜŞÜK BİT

Beş yöntemi aynı eğitim düzeniyle denedik: Larq'ın quantizer'larını ve LittleBit'i PyTorch'ta yeniden yazarak

yöntem	weight	kaynağı	eğitim ayrıntısı
int8	8 bit, kanal başına scale	engine'in bugünkü formatı	eğitimsiz (PTQ)
int4	4 bit, kanal başına scale	standart QAT	STE ile yuvarlama
tern	{−a, 0, +a}	Larq SteTern (TWN)	eşik 0,7 · ort|w|
bin	{−a, +a}, a = ort|w|	Larq MagnitudeAwareSign, SteSign	STE |w| ≤ 1, weight clip
LittleBit	diag(h) sign(U) diag(ℓ) sign(V)ᵀ diag(g)	Lee vd. 2025, Dual-SVID başlangıcı	rank %r, 1 ya da 2 yol

STE (straight-through estimator)

İleri yönde w → sign(w); geri yönde gradient |w| ≤ 1 ise olduğu gibi geçer. Eğitim gerçek değerli bir "gölge" weight'i günceller.

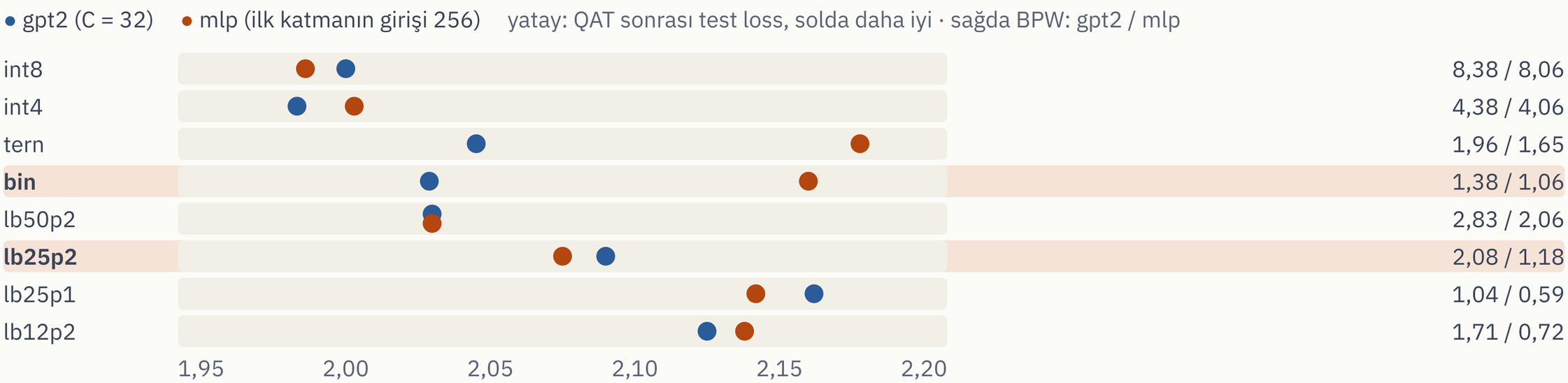
Knowledge distillation

Float model öğretmen; düşük bitli öğrenci, sonraki token dağılımına KL divergence ile yaklaşır. Adam, cosine öğrenme oranı.

Neden TensorFlow değil

Larq'ın quantizer'ları birkaç satırlık fonksiyonlar; PyTorch'ta yazınca modeller ve integer akış aynı kaldı.

Sonuç: transformer’da binary, geniş girişli MLP’de LittleBit daha iyi; INT4 her yerde float kadar iyi



Okuma: 1,4 BPW’de binary gpt2 2,029 (float 2,000); aynı bütçede LittleBit gerisinde. mlp’de ise LittleBit 1,18 BPW’de 2,075, binary 1,06 BPW’de 2,160.

Neden: LittleBit’in scale vektörleri dar katmanlarda weight başına bitin büyük kısmını yiyor

$$\text{BPW} = k \cdot [\, r(d_o + d_i) + 16(d_o + d_i + r) \,] \,/\, (d_o\, d_i)$$

$$\text{binary: } 1 + 16 \,/\, d_i$$

İlk terim işaret bitleri, ikinci terim 16 bitlik h, g ve ℓ scale’leri. *k* yol sayısı, *r* rank. Scale’lerin bedeli *d* ile 1/*d* olarak azalır.

> Makalenin 0,1 BPW sonucu: 4096 × 4096, r = 86, iki yol → 0,0998 BPW. Bu genişlikte scale payı ihmal edilebilir.

katman	binary	yalnız scale	r %25, 1 yol	r %25, 2 yol
32 × 32 (names)	1,50	1,00	1,63	3,25
64 × 64 (stories)	1,25	0,50	1,06	2,13
172 × 64	1,25	0,34	0,71	1,42
4096 × 4096	1,00	0,008	0,51	1,02

Okuma: 32 genişliğinde yalnız scale’ler 1,00 BPW, binary bir katmanın toplam bedelinin üçte ikisi. LittleBit’in bitleri rank yerine scale’lere gidiyor; binary ise aynı bütçeyle tam rank kullanıyor.

7 · DÜŞÜK BİT

Donanım kararı: engine’e 64 hatlı, çarpıcısız bir 1-bit GEMV birimi; binary ve LittleBit aynı birimde çalışıyor

GEMVB (opcode 18)

wbin.mem’deki 64 bitlik bir kelime, bir girişin 64 çıkış için işaretleri (1 = +1, 0 = −1).

64 toplama-çıkarma hattı, DSP yok. 20 bitlik akümülatörler: $|acc| \le 128 \cdot n_in < 2^{19}$.

Seri requant ve issue mantığı INT8 GEMV ile ortak. Kanal scale’i a, $m_0 \cdot 2^{-sh}$ çarpanına katlanır.

SCALE (opcode 17)

Norm’un γ’sı normalde weight’e katlanır: W·diag(γ). Bu binary weight’i bozar.

$$W \cdot \mathrm{diag}(\gamma) = \mathrm{diag}(a) \cdot S \cdot \mathrm{diag}(g),\ S \in \{\pm 1\}$$

diag(a) requant’a gider; g, q, k, v için tek bir SCALE op’u olur: $y_i = \mathrm{requant}(x_i \cdot g_i)$.

LittleBit tek yol

Aynı birimle üç op:

```
SCALE g·γ
GEMVB sign(V)ᵀ, ℓ
GEMVB sign(U), h
```

ℓ ve h ilgili requant’lara katlanır.

Birim yalnız modelde GEMVB varsa sentezlenir (SPECIALIZE). İlk sürüm 14 709 LUT tutuyordu; ayrı 20 bitlik bir banka ile 11 153’e indi (gpt2-bin, tüm kart).

7 · DÜŞÜK BİT

Kartta: düşük bitli modeller 1,7–3,6 kat hızlı ve BRAM'in yarısından azını kullanıyor; bedeli loss

model	cycle / token	hız	BRAM36	LUT	test loss
gpt2 (INT8)	8 713		19,5	7 112	2,002
gpt2-bin	4 983	1,7×	10	11 153	2,028
mlp (INT8)	6 722		14	4 949	1,987
mlp-lb25p1	1 874	3,6×	5	7 940	2,142
stories260K (INT8)	50 397		97,5	9 002	1,291
stories260K-bin	27 992	1,8×	56	12 118	1,927

Hız

64 hat 8 hattan 8 kat geniş; kalan op'lar (norm, attention) değişmediği için toplam kazanç 1,7–3,6 kat.

Bellek ve LUT

BRAM %43–64 azalır; toplama hatları LUT'la kurulduğu için LUT %35–60 artar.

Kalite

names'te +0,03 (binary), +0,16 (LittleBit). stories260K'da +0,64: 260 bin parametre bu görevin sınırında.

8 · DOĞRULAMA

Doğrulama bir zincir: her halka bir öncekinin çıktısını referans alır, kart yerine Mac'te biter

1 · Float ↔ Karpathy
fark < 10^{-4}

2 · Integer ↔ float
loss farkı ≤ 0,022

3 · RTL ↔ integer
tam kart, 18/18 bit-exact

4 · Rastgele testler
655 koşu, ~121 bin token

5 · 4-state
Icarus, X değer yok

6 · Netlist ↔ RTL
kapı seviyesi, 3 araç hatası

7 · Zamanlama
nextpnr, statik analiz

8 · Kartta
yapılmayacak

Neden bit-exact şart

Donanım da referans da aynı integer işlemleri yapıyor. "Yaklaşık aynı" bir hata saklayabilir; tek bir farklı token hatayı gösterir.

Kart olmadan neden yeterli

Tek clock, senkron tasarım: netlist mantıksal olarak doğru ve her yol bir periyoda sığıyorsa çipteki devre simülasyon gibi davranır.

8 · DOĞRULAMA

Tam kart simülasyonu: testbench bir kullanıcı gibi düğmelere basar ve yalnız pinlerden okunabileni karşılaştırır

karşılaştırılan	nasıl
her token	GENTOK satırları ↔ board.py:generate
hash, LED16	kart üstü kontrolün sonucu
ekran metni	80 × 30 karakter, her boşta kalışta
VGA karesi	R, G, B ve senkron pinlerinden
7-segment, LED	token sayacı, hız göstergesi
pin imzası	her cycle'da bütün pinler

Model başına 4 koşu: otomatik sampling, BTNC, SW0 açıkken BTNU (greedy), BTND (temizle). Girişler düşen kenarda sürülür, çıkışlar düşen kenarda örneklenir.

stories260K, gerçek çıktı

```
run 0: 256 tokens (identical)
hash 44fe319f = 44fe319f
screen, status, check, LEDs,
7-segment OK
VGA frame: 0 sync errors,
0 pixels differ
```

bigram	mlp	rnn
gru	bow	gpt2
microgpt	llama	nanochat
stories260k	ts_gpt2	ts_gru
ts_llama	ts_microgpt	ts_nanochat
gpt2-bin	mlp-lb25p1	stories-bin

18 / 18 geçti, toplam 11 241 token.

8 · DOĞRULAMA

Rastgele ve köşe durumu testleri, normal kullanımda görünmeyen iki gerçek RTL hatası buldu

655

rastgele koşu

~121 bin token; koşu ortasında düğme, CPU_RESET; 32 sampling ayarının 30–32'si.

16

köşe durumu

koşu sırasında basılan düğme kuyruğa girer; reset sonrası token'lar referansın öneki.

0 X

4-state simülasyon

Icarus, gru 2,7 M cycle: pinlerde ve metin RAM'inde bilinmeyen değer yok.

Hata 1: kaybolan düğme

Kuyruktaki bir koşunun başladığı cycle'a denk gelen BTNU basışı kayboluyordu. Kuyruğa alma mantığı düzeltildi.

Hata 2: reset'siz sayaç

Heartbeat sayacının reset'i yoktu: 2-state Verilator'da görünmüyordu, 4-state simülasyon LED0'da X gösterdi. CPU_RESET onu sıfırlamıyordu.

8 · DOĞRULAMA

Netlist simülasyonu: RTL doğru olsa bile araçlar hata yapabilir; üç araç hatası bulduk

sv2v 0.0.13

```
64'(att_a[j]) >>> 32
```

Signed bir eleman için boyut cast'ini işaretsiz çeviriyor; aritmetik kaydırma mantıksala dönüyor, negatifler +127'ye doyuyordu.

Çözüm: açık $signed(...).

Yosys 0.69: wreduce + peepopt

```
va_d[8*(((h<<3)+d) % P) +: 8]
```

İki optimizasyon art arda çalışınca q·k toplamındaki dinamik bayt seçimi bozuluyordu; t = 0'dan sonra attention yanlış.

Çözüm: önce head dilimi, sonra sabit indeks.

Yosys 0.69: DSP48E1

```
prod_r <= sum * m0_s;
```

Registerlı girişli requant çarpımını DSP'ye eşlerken işaret uzatmasını düşürüyor; negatif akümülatörler +max'a doyuyor.

Çözüm: netlist ve açık kaynak bitstream'de çarpıcılar LUT'ta (-nodsp).

Sonuç: gerçek üst modülün Yosys netlist'i (MMCM, BUFG, BRAM) gru, gpt2-bin ve mlp-lb25p1'de her pinde, her cycle RTL ile aynı. gpt2-bin: 4 200 047 cycle, VGA karesi aynı. Verilator ve Vivado SystemVerilog'u doğrudan okuduğu için ilk iki hatadan etkilenmiyordu.

9 · YERLEŞİM VE HIZ

Vivado olmadan, Mac’te açık kaynak bir akışla: sentez, yerleşim, yönlendirme, zamanlama ve bitstream

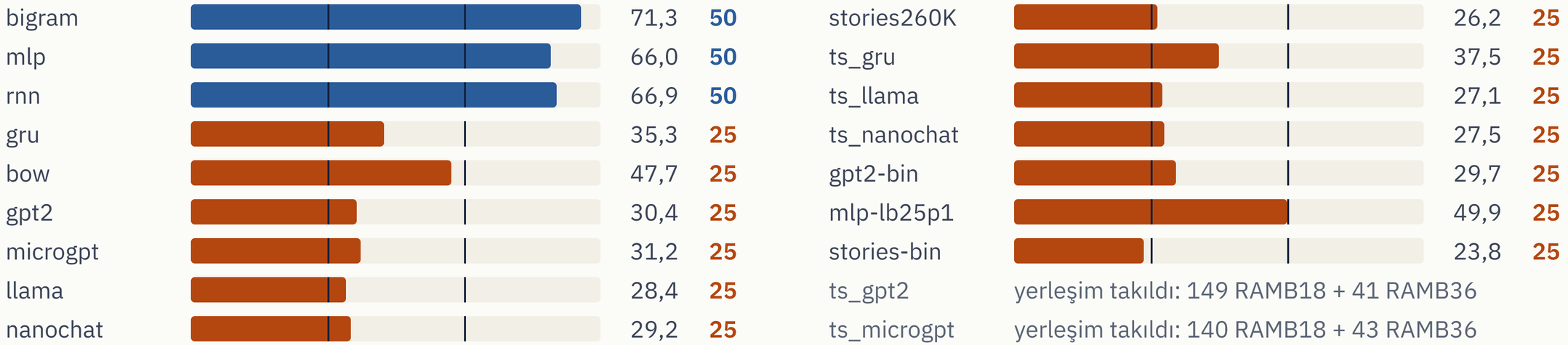


sorun	çözüm
macOS SDK’sı Xcode linker’ıyla okunamıyor	derlemede Xcode’un kendi SDK’sı (SDKROOT)
nextpnr pin adlarını boş görüyor	PACKAGE_PIN boşalmış bir referanstan kopyalanıyor; XDC’de doğrudan LOC
50 MHz tutmayınca akış duruyor	zamanlama raporu her durumda yazılır; tutmazsa 25 MHz’te yeniden kurulur
DSP eşlemesinde Yosys hatası	çarpıcılar LUT’ta (-nodsp): netlist simülasyonuyla bit-exact kanıtlı netlist

bigram uçtan uca 1 dakika (3,8 MB bitstream); stories260K, 50 MHz denemesi ve 25 MHz’e inişle ~13 dakika.

Fmax: makemore'un üç modeli 50 MHz'i tutuyor; transformer'lar 26–31 MHz'te kalıyor ve 25 MHz'te çalışıyor

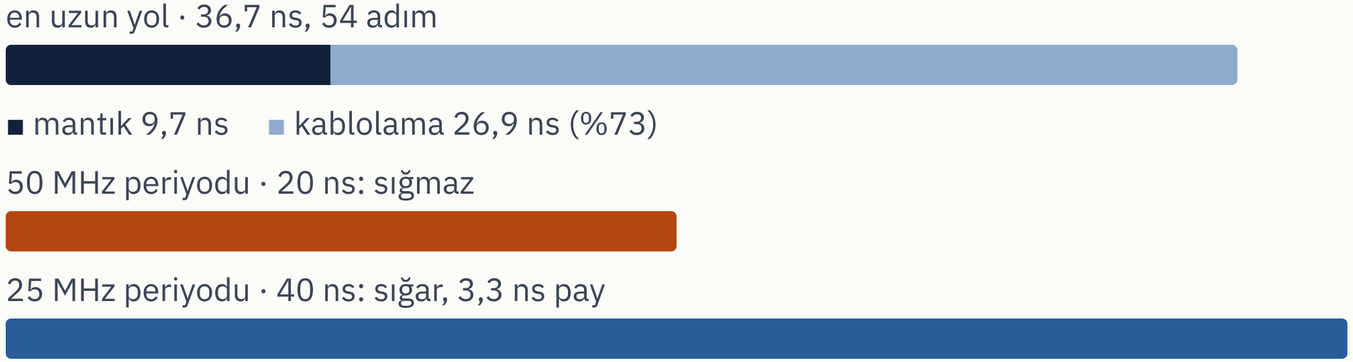
■ 50 MHz'te tuttu ■ tutmadı, 25 MHz'te kuruldu çizgiler: 25 ve 50 MHz · çubuk: 50 MHz denemesindeki Fmax · sağda çalışma clock'u



Bulgu: 16 modelin hepsi çalıştığı clock'ta zamanlamayı tutuyor. Fmax yerleşim tohumuna göre oynuyor: bow'un 25 MHz kurulumu bir tohumla 50,1, ötekiyle 44,0 MHz verdi.

Kritik yol: oku, çarp, yaz tek cycle’da; çarpmalar LUT’ta olunca bu 37 ns sürüyor

j, f_n0 → va_a: tampon adresi → LUTRAM oku, qh → LUT çarpıcı, carry → vw_d: yaz



Model değişir, kalıp aynı

stories260K, llama, nanochat: attention’ın head dilimi (qh). gpt2-bin: NORM çıkışı, d · inv çarpımı ve değişken kaydırma.

Çare

Okuma ile hesap arasına bir register: yol ikiye bölünür, op’lar bir cycle gecikir. Hedef 50 MHz.

Yerleşimden sonra kaynaklar: en büyük model bile çipin üçte birinden azını kullanıyor

model	LUT bel	FF	CARRY4	BRAM36	DSP
bigram	6 362 (%5)	1 334	242	2 (%1)	0
mlp	13 631 (%11)	2 228	415	14 (%10)	0
gru	15 506 (%12)	2 255	479	16 (%12)	0
gpt2	29 435 (%23)	3 265	773	19,5 (%14)	0
nanochat	32 288 (%25)	3 416	840	16 (%12)	0
stories260K	34 643 (%27)	3 513	862	97,5 (%72)	0
ts_gru	15 826 (%12)	2 289	475	69,5 (%51)	0
gpt2-bin	35 041 (%28)	5 958	1 106	10 (%7)	0
stories260K-bin	39 334 (%31)	6 120	1 198	56 (%41)	0

LUT bel: 126 800 yuvadan. BRAM36 = RAMB36 + RAMB18/2, Yosys tahminiyle aynı. DSP 0: çarpıcılar LUT’ta; Yosys’in DSP’li tahmini stories260K için 9 002 LUT ve 66 DSP.

Gerçek hız = clock / (cycle / token): stories260K kartta saniyede 496 token yazıyor, 256 token'lık hikâye yarım saniye

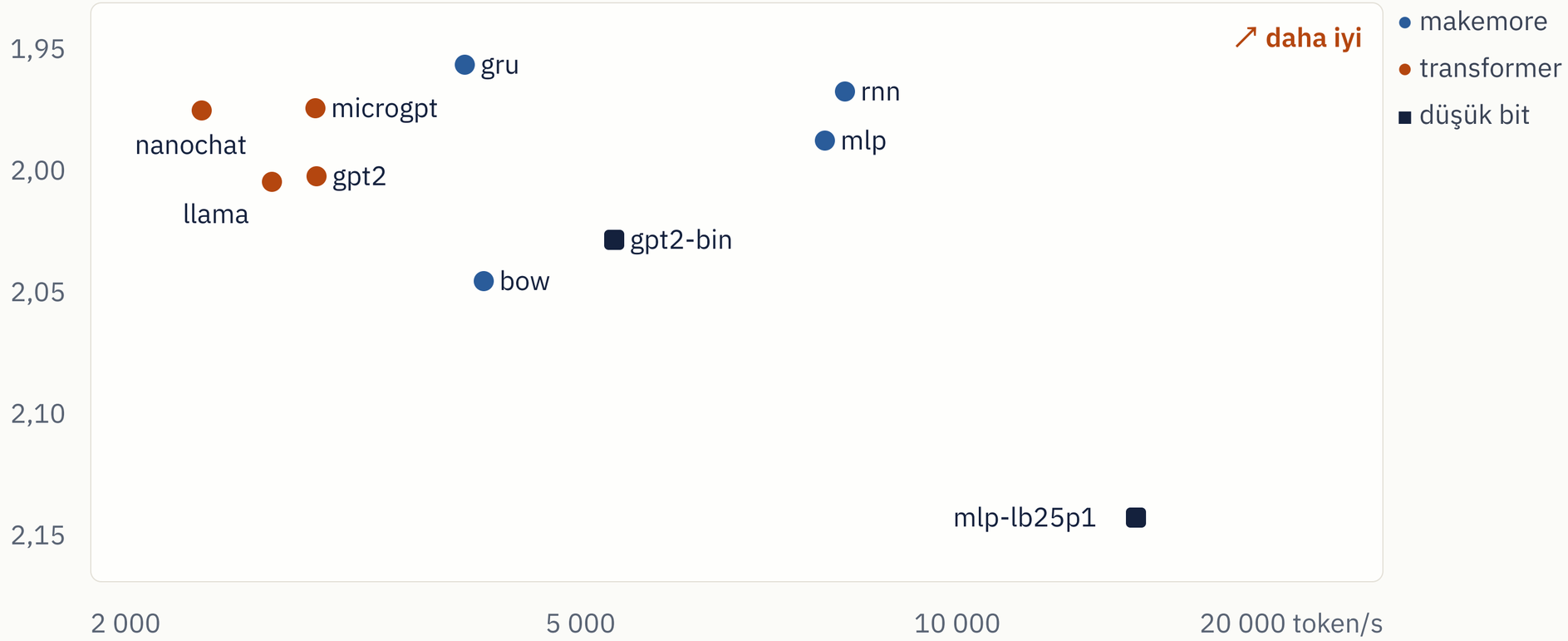
model	MHz	cycle/token	token/s
bigram	50	157	318 066
mlp	50	6 722	7 438
rnn	50	6 464	7 735
gru	25	6 605	3 785
bow	25	6 368	3 926
gpt2	25	8 713	2 869
microgpt	25	8 746	2 858
llama	25	9 491	2 634

model	MHz	cycle/token	token/s
nanochat	25	10 813	2 312
stories260K	25	50 397	496
ts_gru	25	30 407	822
ts_llama	25	50 397	496
ts_nanochat	25	50 680	493
gpt2-bin	25	4 983	5 017
mlp-lb25p1	25	1 874	13 340
stories-bin	25	27 992	893

Ölçek için: bir insan saniyede birkaç token okur; kart en yavaş modelde bile okumaktan yüz kat hızlı yazıyor. Bu yüzden kartta bir de saniyede ~25 token'lık daktilo modu var. Pipeline ile 50 MHz'e çıkılırsa bütün 25 MHz sayıları iki katına çıkar.

10 · SONUÇ

Kalite ve hız birlikte: names’te en iyi loss gru’da, en hızlı INT8 model rnn; LittleBit hızı loss ile ödüyor



10 · SONUÇ

Sınırlar: neyi kanıtladık, neyi henüz kanıtlamadık

Fiziksel kart yok

Kanıt simülasyon ve zamanlama analizi. Yerleşimden sonraki son devre simüle edilemiyor; nextpnr onu Verilog olarak yazamıyor.

Fmax yaklaşık

nextpnr'ın gecikme modeli topluluk tarafından çıkarıldı; Vivado'nun sonucu farklı olabilir. Çarpıcılar LUT'ta, yani Vivado'nun DSP'li devresinden yavaş ve büyük.

Kapasite

607 KB BRAM, ~300 bin parametreye kadar yetiyor. nanoGPT'nin ve llm.c'nin hazır checkpoint'leri (10 M+) için DDR2 ve farklı bir mimari gerekir.

Eksik parçalar

ts_gpt2 ve ts_microgpt yerleşemedi. gpt-nano sıralama demosu bu kart tasarımında yok. Larq paketiyle sayısal karşılaştırma ve iki yollu LittleBit donanımı yapılmadı.

10 · SONUÇ

Sonraki adımlar: önce hız, sonra doğrulamanın son boşluğu, sonra kapsam

1. **Engine’e pipeline:** tampon okuma ile hesap arasına register. Hedef 50 MHz, yani token/s’de 2 kat. 18 modelin simülasyonları yeniden geçmeli.

2. **Yerleştirilen netlist’in simülasyonu:** 18 modelin hepsinde, gerçek clock ayarı ve gerçek debounce süreleriyle.

3. **DSP’ler ve bellek düzeni:** Yosys hatasını raporlamak, requant dışındaki çarpıcıları DSP’ye koymak; ts_gpt2 ve ts_microgpt’nin tablolarını RAMB36’ya sığacak şekilde bölmek.

4. **19. model:** minGPT’nin gpt-nano sıralama demosu; ekranda girdi dizisi ve sıralanmış çıktı.

5. **Düşük bit:** stories260K için daha uzun binary QAT; iki yollu LittleBit’in donanımda denenmesi.

ÖZET

1 · Karpathy’nin dokuz mimarisi ve stories260K, tek bir mikrokodlu integer engine’de; modeli değiştiren yalnız BRAM’deki mikrokod ve weight’ler.

2 · Kart bilgisayarsız çalışıyor: üretim döngüsü, integer sampling, metin ve VGA ekranı FPGA’de; 18 modelin hepsi Python referansıyla bit bit aynı.

3 · Binary weight’ler 1,7–3,6 kat hız ve yarı bellek getiriyor; LittleBit küçük modellerde ancak geniş katmanlarda binary’yi geçiyor.

4 · Açık kaynak akışla bitstream ve Fmax Mac’te: 16 model yerleşti; makemore’un üçü 50 MHz’te, diğerleri 25 MHz’te tutuyor. Engine pipeline’ı ile hedef 50 MHz.

Teşekkürler. Sorular?

github.com/Muratatceken/mingpt-fpga